

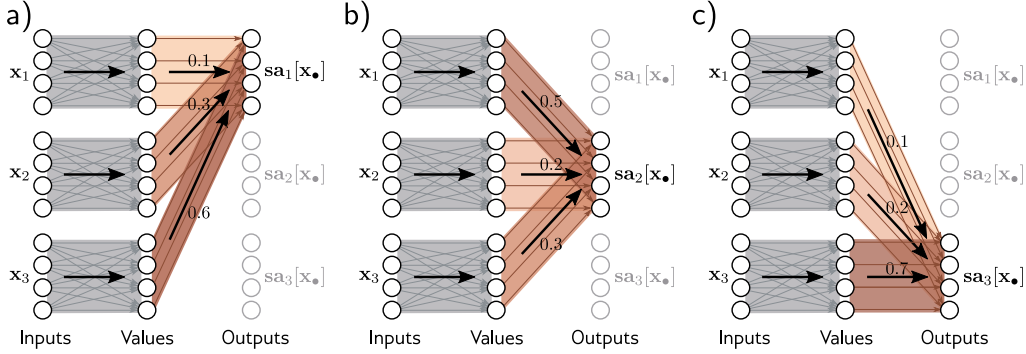


Figure 12.1 Self-attention as routing. The self-attention mechanism takes N inputs $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^D$ (here $N = 3$ and $D = 4$) and processes each separately to compute N value vectors. The n^{th} output $\mathbf{sa}_n[\mathbf{x}_1, \dots, \mathbf{x}_N]$ (written as $\mathbf{sa}_n[\mathbf{x}_\bullet]$ for short) is then computed as a weighted sum of the N value vectors, where the weights are positive and sum to one. a) Output $\mathbf{sa}_1[\mathbf{x}_\bullet]$ is computed as $a[\mathbf{x}_1, \mathbf{x}_1] = 0.1$ times the first value vector, $a[\mathbf{x}_2, \mathbf{x}_1] = 0.3$ times the second value vector, and $a[\mathbf{x}_3, \mathbf{x}_1] = 0.6$ times the third value vector. b) Output $\mathbf{sa}_2[\mathbf{x}_\bullet]$ is computed in the same way, but this time with weights of 0.5, 0.2, and 0.3. c) The weighting for output $\mathbf{sa}_3[\mathbf{x}_\bullet]$ is different again. Each output can hence be thought of as a different routing of the N values.

12.2.1 Computing and weighting values

Equation 12.2 shows that the same weights $\mathbf{\Omega}_v \in \mathbb{R}^{D \times D}$ and biases $\beta_v \in \mathbb{R}^D$ are applied to each input $\mathbf{x}_\bullet \in \mathbb{R}^D$. This computation scales linearly with the sequence length N , so it needs fewer parameters than a fully connected network relating all DN inputs to all DN values. In fact, the value computation can be viewed as a sparse matrix operation with shared parameters that relates these DN quantities (figure 12.2b).

The attention weights $a[\mathbf{x}_m, \mathbf{x}_n]$ combine the values from different inputs. They are also sparse since there is only one weight for each ordered pair of inputs $(\mathbf{x}_m, \mathbf{x}_n)$, regardless of the size of these inputs (figure 12.2c). It follows that the number of attention weights has a quadratic dependence on the sequence length N , but is independent of the length D of each input.

12.2.2 Computing attention weights

In the previous section, we saw that the outputs result from two chained linear transformations; the value vectors $\beta_v + \mathbf{\Omega}_v \mathbf{x}_m$ are computed independently for each input $\mathbf{x}_m$, and these vectors are combined linearly by the attention weights $a[\mathbf{x}_m, \mathbf{x}_n]$. However, the overall self-attention computation is *nonlinear*. As we'll see shortly, the attention weights are themselves nonlinear functions of the input. This is an example of a *hyper-network*, where one network branch computes the weights of another. To compute the

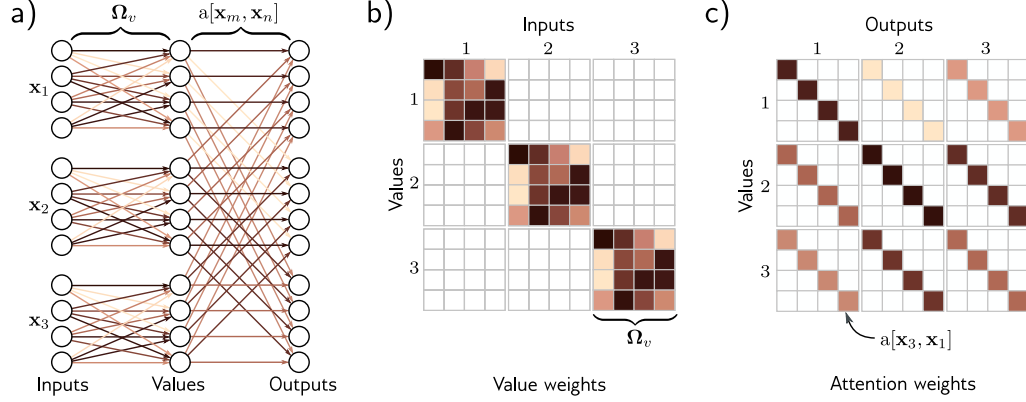


Figure 12.2 Self-attention for $N=3$ inputs $\mathbf{x}_n$, each with dimension $D=4$. a) Each input $\mathbf{x}_m$ is operated on independently by the same weights Ω_v (same color equals same weight) and biases β_v (not shown) to form the values $\beta_v + \Omega_v \mathbf{x}_m$. Each output is a linear combination of the values, with the attention weight $a[\mathbf{x}_m, \mathbf{x}_n]$ defining the contribution of the m^{th} value to the n^{th} output. b) Matrix showing block sparsity of linear transformation Ω_v between inputs and values. c) Matrix showing sparsity of attention weights relating values and outputs.

attention, we apply two more linear transformations to the inputs:

$$\begin{aligned} \mathbf{q}_n &= \beta_q + \Omega_q \mathbf{x}_n \\ \mathbf{k}_m &= \beta_k + \Omega_k \mathbf{x}_m, \end{aligned} \quad (12.4)$$

where $\{\mathbf{q}_n\}$ and $\{\mathbf{k}_m\}$ are termed *queries* and *keys*, respectively. Then we compute **dot products** between the queries and keys and pass the results through a softmax function:

$$\begin{aligned} a[\mathbf{x}_m, \mathbf{x}_n] &= \text{softmax}_m [\mathbf{k}_\bullet^T \mathbf{q}_n] \\ &= \frac{\exp [\mathbf{k}_m^T \mathbf{q}_n]}{\sum_{m'=1}^N \exp [\mathbf{k}_{m'}^T \mathbf{q}_n]}, \end{aligned} \quad (12.5)$$

so for each $\mathbf{x}_n$, they are positive and sum to one (figure 12.3). For obvious reasons, this is known as *dot-product self-attention*.

The names “queries” and “keys” were inherited from the field of information retrieval and have the following interpretation: the dot product operation returns a measure of similarity between its inputs, so the weights $a[\mathbf{x}_\bullet, \mathbf{x}_n]$ depend on the relative similarities between the n^{th} query and all of the keys. The softmax function means that the key vectors “compete” with one another to contribute to the final result. The queries and keys must have the same dimensions. However, these can differ from the dimension of

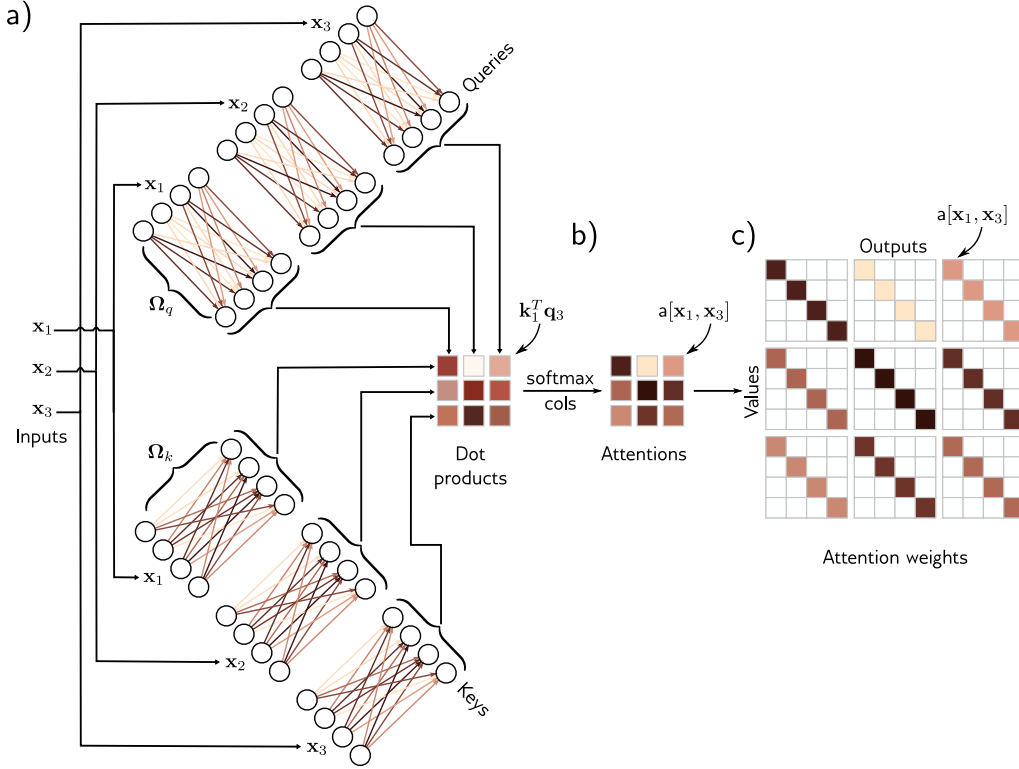


Figure 12.3 Computing attention weights. a) Query vectors $\mathbf{q}_n = \beta_q + \Omega_q \mathbf{x}_n$ and key vectors $\mathbf{k}_n = \beta_k + \Omega_k \mathbf{x}_n$ are computed for each input $\mathbf{x}_n$. b) The dot products between each query and the three keys are passed through a softmax function to form non-negative attentions that sum to one. c) These route the value vectors (figure 12.1) via the sparse matrix from figure 12.2c.

the values, which is usually the same size as the input, so the representation doesn't change size.

Problems 12.1–12.2

12.2.3 Self-attention summary

The n^{th} output is a weighted sum of the same linear transformation $\mathbf{v}_\bullet = \beta_v + \Omega_v \mathbf{x}_\bullet$ applied to all of the inputs, where these attention weights are positive and sum to one. The weights depend on a measure of similarity between input $\mathbf{x}_n$ and the other inputs. There is no activation function, but the mechanism is nonlinear due to the dot-product and a softmax operation used to compute the attention weights.

Note that this mechanism fulfills the initial requirements. First, there is a single shared set of parameters $\phi = \{\beta_v, \Omega_v, \beta_q, \Omega_q, \beta_k, \Omega_k\}$. This is independent of the

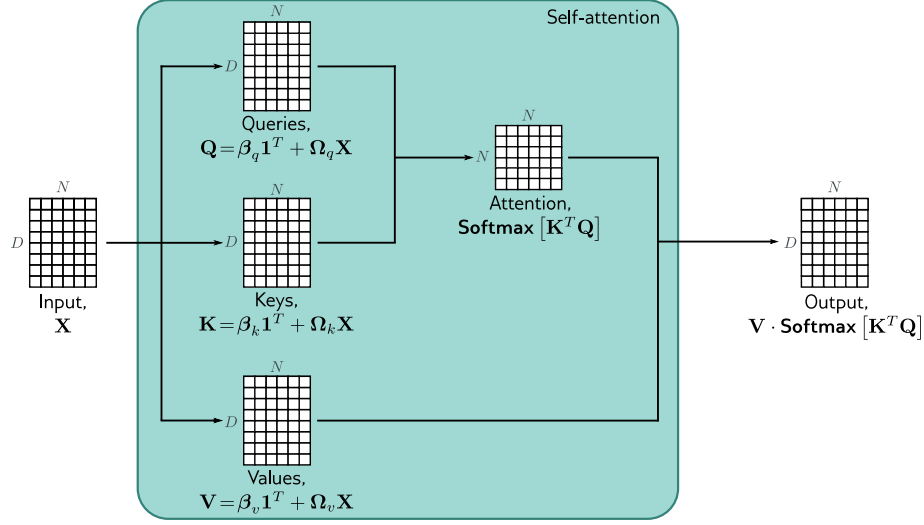


Figure 12.4 Self-attention in matrix form. Self-attention can be implemented efficiently if we store the N input vectors $\mathbf{x}_n$ in the columns of the $D \times N$ matrix $\mathbf{X}$. The input $\mathbf{X}$ is operated on separately by the query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$, and value matrix $\mathbf{V}$. The dot products are then computed using matrix multiplication, and a softmax operation is applied independently to each column of the resulting matrix to calculate the attentions. Finally, the values are post-multiplied by the attentions to create an output of the same size as the input.

number of inputs N , so the network can be applied to different sequence lengths. Second, there are connections between the inputs (words), and the strength of these connections depends on the inputs themselves via the attention weights.

12.2.4 Matrix form

The above computation can be written in a compact form if the N inputs $\mathbf{x}_n$ form the columns of the $D \times N$ matrix $\mathbf{X}$. The values, queries, and keys can be computed as:

$$\begin{aligned} \mathbf{V}[\mathbf{X}] &= \beta_v \mathbf{1}^T + \Omega_v \mathbf{X} \\ \mathbf{Q}[\mathbf{X}] &= \beta_q \mathbf{1}^T + \Omega_q \mathbf{X} \\ \mathbf{K}[\mathbf{X}] &= \beta_k \mathbf{1}^T + \Omega_k \mathbf{X}, \end{aligned} \tag{12.6}$$

where $\mathbf{1}$ is an $N \times 1$ vector containing ones. The self-attention computation is then:

$$\text{Sa}[\mathbf{X}] = \mathbf{V}[\mathbf{X}] \cdot \text{Softmax}[\mathbf{K}[\mathbf{X}]^T \mathbf{Q}[\mathbf{X}]], \tag{12.7}$$

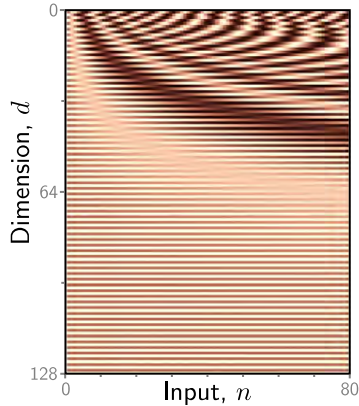


Figure 12.5 Positional encodings. The self-attention architecture is equivariant to permutations of the inputs. To ensure that inputs at different positions are treated differently, a positional encoding matrix $\mathbf{\Pi}$ can be added to the data matrix. Each column is different, so the positions can be distinguished. Here, the position encodings use a predefined procedural sinusoidal pattern (which can be extended to larger values of N if necessary). However, in other cases, they are learned.

where the function $\text{Softmax}[\bullet]$ takes a matrix and performs the softmax operation independently on each of its columns (figure 12.4). In this formulation, we have explicitly included the dependence of the values, queries, and keys on the input $\mathbf{X}$ to emphasize that self-attention computes a kind of triple product based on the inputs. However, from now on, we will drop this dependence and just write:

$$\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}[\mathbf{K}^T \mathbf{Q}]. \quad (12.8)$$

[Notebook 12.1](#)
[Self-attention](#)

12.3 Extensions to dot-product self-attention

In the previous section, we described self-attention. Here, we introduce three extensions that are almost always used in practice.

12.3.1 Positional encoding

Observant readers will have noticed that the self-attention mechanism overlooks important information: the computation does not take into account the order of the inputs $\mathbf{x}_n$. More precisely, it is equivariant with respect to input permutations. However, order *is* important when the inputs correspond to the words in a sentence. The sentence [The woman ate the raccoon](#) has a different meaning than [The raccoon ate the woman](#). There are two main approaches to incorporating position information.

[Problem 12.3](#)

Absolute positional encodings: A matrix $\mathbf{\Pi}$ is added to the input $\mathbf{X}$ that encodes positional information (figure 12.5). Each column of $\mathbf{\Pi}$ is unique and hence contains information about the absolute position in the input sequence. This matrix can be chosen by hand or learned. It may be added to the network inputs or at every network layer. Sometimes it is added to $\mathbf{X}$ in the computation of the queries and keys but not to the values.